

UNIVERSIDAD NACIONAL DE AVELLANEDA



MATERIA:

Sistema de Control Automático

Trabajo de Laboratorio II

Integrantes:

Apellido y Nombres

E-mails

Rodriguez Cioffi, Julian

julianrc999@hotmail.com

Pachecho, Nahuel

pacheconahuel100@gmail.com

Hernández, Leonardo

leoehz4@gmail.com

Valdivia, Ramón Antonio

ramon.valdivia@alu.bue.edu.ar /
antoniovaldivia42@hotmail.com

Profesores: Caporaletti, Guillermo

Índice

Índice	2
Objetivos	3
Preparación	3
Introducción	4
Desarrollo	4
Componentes utilizados	4
Modelización	5
Selección de parámetros del controlador proporcional	6
Selección de parámetros del controlador PI	7
Experimento	10
Primera Experiencia	10
Segunda Experiencia	10
Conclusiones	15
Bibliografía	16
Anexo	17
Modelo de retardo de Maclaurin	17
Código Arduino del sensor de temperatura	18

Objetivos

- Controlar un sistema físico con una acción de control proporcional y proporcional-integral.
- Elegir los parámetros más adecuados para configurar nuestro controlador teniendo en cuenta: tiempo de respuesta, precisión y sobrevalor. Evaluar su comportamiento real a partir de una experiencia.

Preparación

- Precisar o mejorar nuestro modelo del sistema a partir de nuestra práctica previa con control sí-no.
- Evaluar e implementar posibles mejoras a nuestro sistema físico y a nuestro actuador.
- Analizar la forma de implementar un control de potencia variable con este actuador.
- Evaluar los problemas y/o errores de nuestro sensor.
- Elegir una (o varias) constante K_P y simular nuestro sistema. Justificar nuestro parámetro final elegido. De ser necesario graficar el Lugar Geométrico de las Raíces y analizarlo.
- Precisar el margen de fase y de ganancia de nuestro sistema para el valor de K_P elegido.
- Elegir una (o varias) constante de integración T_i . Graficar el Lugar Geométrico de las Raíces, analizarlo y elegir T_i y K_P . Precisar el margen de fase y de ganancia de nuestro sistema para los parámetros elegidos. En caso de no ser conveniente utilizar un control PI, justificarlo debidamente.
- Elaborar los códigos necesarios para Arduino.

Introducción

El presente trabajo tiene como objetivo partir de nuestra experiencia modelizada anteriormente la cual consto en controlar una heladera de 12V, pero ahora modificaremos el control On-Off por un control Proporcional-Integral (de ahora en más referido como PI) por lo que buscaremos ajustar los parámetros (K_p y T_i) de tal forma que optimicemos el modelo para que demore el menor tiempo posible y a su vez eliminemos el error de estado estacionario producido por el control proporcional al agregarle una componente integral.

Una vez hallados los parámetros de nuestro modelo pasaremos a la segunda etapa en donde realizaremos la experiencia real para evaluar si nuestra modelización es capaz de asemejarse a la realidad.

En nuestra experiencia previa obtuvimos los valores para la Resistencia Térmica y la Capacidad Calorífica y terminamos adoptando que nuestra planta modelizada como un sistema de primer orden nos quedó:

$$G(s) = \frac{1.64}{1+617.28s}$$



Al cual tuvimos que agregarle un retardo modelizado por MacLaurin (consultar anexo [Modelo de Retardo de MacLaurin](#)) para que la simulación fuera más acorde con la simulación con lo cual finalmente terminamos obteniendo:

$$H(s) = \frac{1.64}{1+617.28s} \cdot \frac{1}{1+125s+7812.5s^2}$$

Este será nuestro punto de partida a la hora de encarar esta nueva experiencia.

Desarrollo

Componentes utilizados

Los componentes que utilizamos son los que se detallan a continuación.

- La heladera de 12V contiene una celda de Peltier, la cual sirve de refrigeración para nuestro sistema.
- Dos sensores de temperatura son los encargados de darnos los parámetros de temperatura en la heladera y la temperatura ambiente.
- El Arduino se encarga de la lógica de control sobre el sistema, efectuando las mediciones y el encendido/apagado.
- Protoboard
- Fuente de tensión regulable
- Transistores, configuración darlington

Modelización

Como mencionamos anteriormente, nuestro punto de partida va a ser la planta que definimos en nuestra primer experiencia, a la cual le cambiaremos el control On-Off para agregarle uno PI como muestra la Figura 1

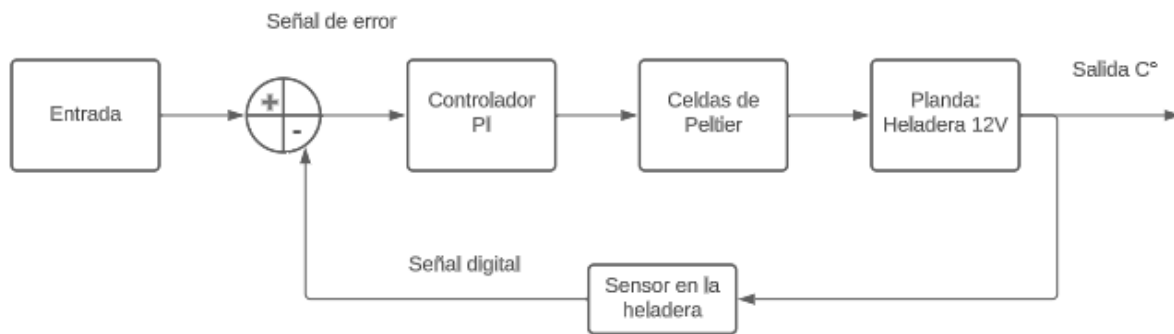


Fig. 1 Diagrama en bloques del sistema con control PI.

El motivo por el cual escogimos un control PI, es debido a que, si bien el sistema proporcional nos permitía poder reducir drásticamente el tiempo de establecimiento sobre la temperatura con un margen de 1 °C, el mismo no llegaba al valor objetivo deseado, teniendo una diferencia considerable de temperatura objetivo, por lo cual, decidimos agregar un factor integrador, el cual nos permitía poder acercarnos mucho más al valor de temperatura deseada.

La compensación evita que se produzca el efecto enrole. El efecto enrole se produce al agregar una saturación de alimentación de un sistema a un integrador. Lo que sucede es que, ante un cambio brusco en el momento en el que se satura el sistema, el bloque integrador aún continúa integrando, aún más de lo que debería o cuando no debería, por lo que la salida de la transferencia termina por tener un sobrevalor mayor al esperado. En este punto, el bloque integrador termina por querer compensar el sobrevalor integrando con signo opuesto invirtiendo la pendiente hasta poder llegar al valor deseado u objetivo de salida, provocando cierta inestabilidad y aumentando el tiempo de estabilización del mismo. Este efecto se puede evitar configurando los límites de integración.

Si bien en un sistema PID, el componente derivativo ayudaría a reducir el sobrevalor, en nuestro caso no fue necesario ya que se adecuaba a lo que queríamos, con un sobrevalor menor al 5%.

Selección de parámetros del controlador proporcional

Para la obtención de los parámetros del control proporcional lo primero que hicimos fue utilizar la herramienta SciLab para establecer nuestra planta $G(s)$ y obtener cuanto es la ganancia máxima que podríamos darle al sistema antes de que este se vuelva inestable (es decir, que la parte real de los polos pase a ser positiva) por lo que graficamos un Evans y movemos el cursor hasta localizar ese punto crítico llegando así que la máxima ganancia soportada es de 7.36 el cual comprobamos utilizando la consola de SciLab con la función `kpure(G)`

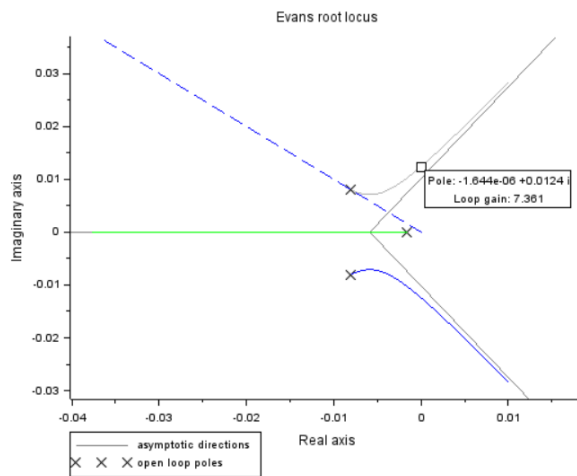


Fig 2. Evans de nuestra planta

```
--> kpure(G)
ans =

7.3652325
```

Tras esto decidimos escoger tener un margen de ganancia amplio (mayor a 6) en caso de que la modelización no contemple alguna característica del experimento, tener un margen de seguridad.

Por lo tanto:

$$K_p = \frac{7,36}{6,57} = 1,12$$

Únicamente con este valor de K_p se obtuvo una respuesta simulada con un sobrevalor de 4,3% y con un error de estado estacionario del 35%.

Esto ya nos da un punto de partida sobre que valor de K_p podríamos utilizar para nuestro controlador PI.

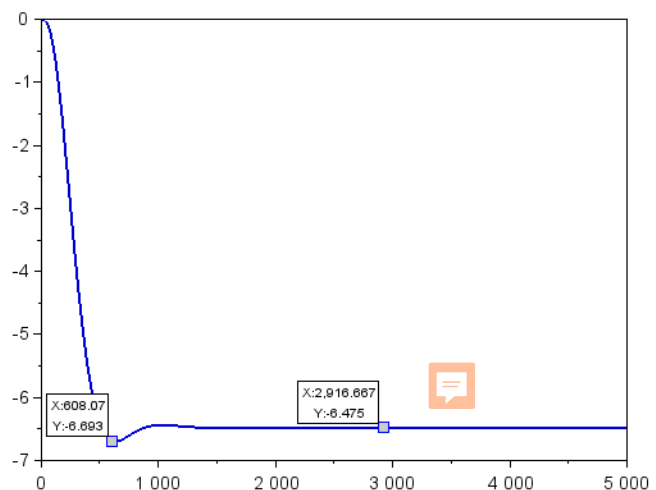


Fig 3. Simulación con Control Proporcional K_p de 1.12

Selección de parámetros del controlador PI

Ya teniendo nuestra planta modelizada y habiendo realizado una primer simulación para el control proporcional pasamos a implementar un controlador PI obtenido en donde pondremos a prueba el K_p seleccionado en la sección anterior para ver cómo se comporta nuestro sistema por lo cual procedemos a realizar la simulación en SciLab para lo cual utilizaremos el fragmento de código mostrado en la Fig. 4 en donde establecemos nuestra planta, el retardo y el valor de nuestros parámetros para simular el comportamiento de nuestra planta

```
1 s=%s
2 G=1.67/(657.6*s+1)
3
4
5 R=1/(1+s*125+s^2*7812.5)
6
7 G=G*R
8 G=syslin('c',G)
9 K=1.12
10 // Pruebas variando el tiempo de integración ti
11 close()
12 close()
13 figure(1)
14 t3=0:1:5000
15 for ti=[650]
16     PI=K*(1+1/(ti*s))
17     T1=(PI*G)/(1+PI*G)
18     Y=csim('step',t3,T1)*(-15)
19     plot(t3,Y)
20 end
```

Fig 4. Código utilizado para la simulación

a lazo cerrado obteniendo así el resultado de la Figura 5

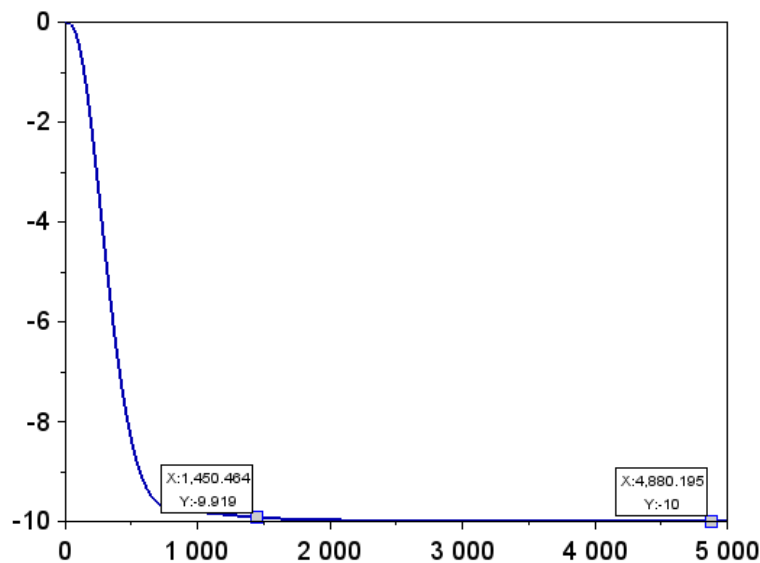


Fig 5. Simulación con control PI. K_p de 1,12 y T_i de 650

Por lo que según la simulación, nuestro sistema tendrá un tiempo de establecimiento de 567 segundos, o 9 minutos y 27 segundos, para un margen de 1 °C (10%) de diferencia de la temperatura objetivo para 10 grados, o de 626,5 segundos (10 minutos y 26 segundos) para 15 °C.

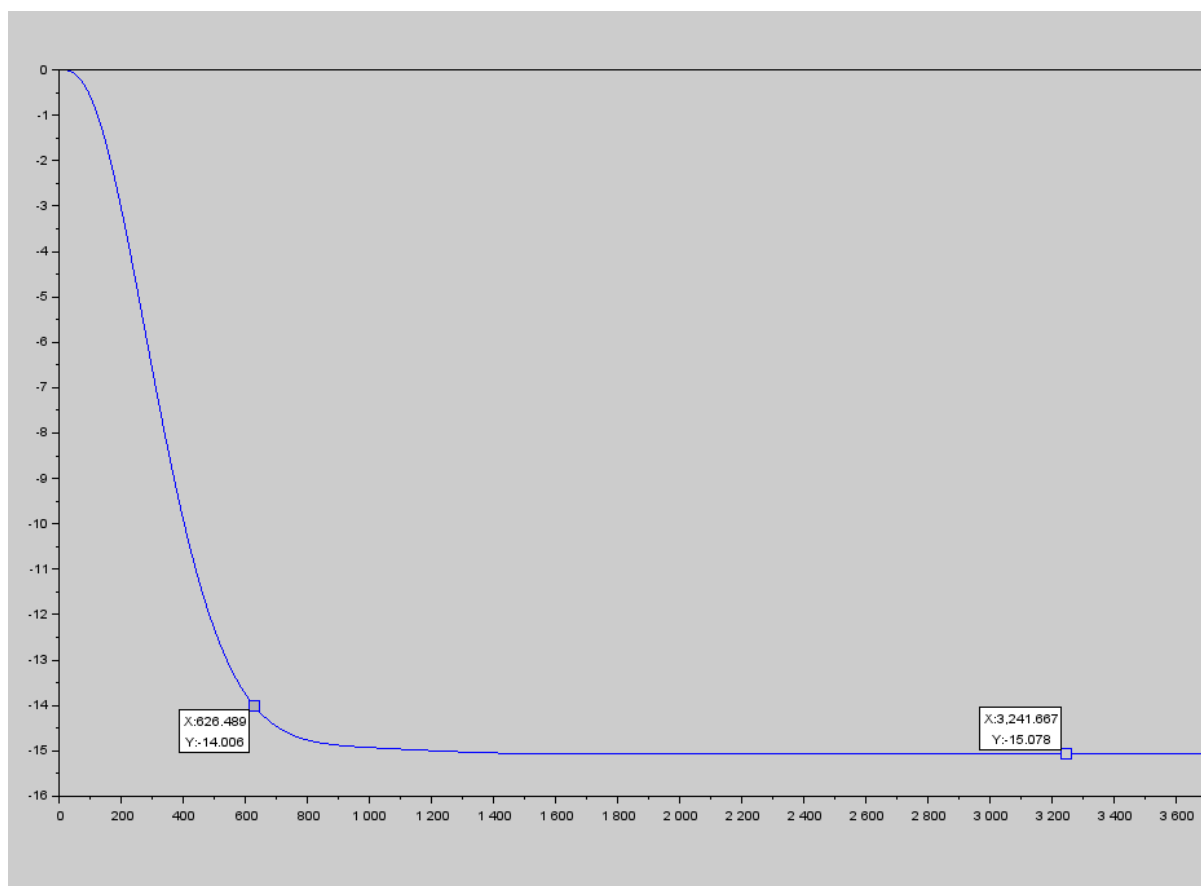


Fig 6. Simulación del sistema para ver el tiempo de establecimiento.

Habiendo simulado con varios tiempos de integración mediante iteración llegamos a escoger un tiempo de integración ($T_i = 650$) tal que corrige el error de estado estacionario y por otro lado hace que ya no tenga sobrevalor.

Ya para este punto nuestro sistema final a nivel transferencia nos quedó establecido con la siguiente configuración:

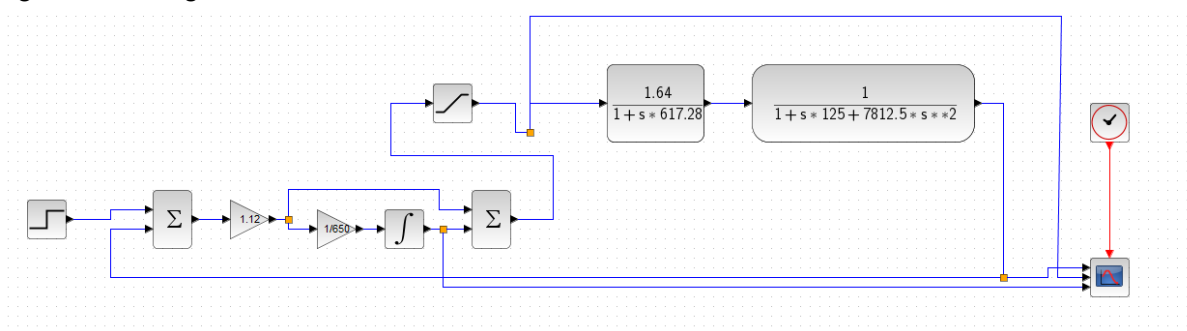


Fig. 7 Elementos utilizados para simular en xcos

Donde la señal escalón corresponde a nuestro objetivo final de temperatura, y donde agregamos el bloque proporcional e integrador con el fin de utilizar un sistema proporcional integral. Cabe destacar que agregamos un bloque saturador para imitar la realidad, donde nuestra salida satura a una alimentación de 12V.

Control Proporcional	Tiempo de integración [segundos]	Sobrevalor relativo	Tiempo de establecimiento (10%) [segundos]
0.5	100	60%	3700
1.12	200	47%	1617
2.45	400	44%	990
4.8	600	68%	3450
2.45	650	23%	914
1.12	650	0%	655

Tabla 1: valores obtenidos en simulaciones ante una entrada escalón de -10

Control Proporcional	Tiempo de integración [segundos]	Sobrevalor relativo	Tiempo de establecimiento (10%) [segundos]
0.5	100	29%	2800
1.12	200	24%	1230
2.45	400	19%	1073
4.8	600	15%	919
2.45	650	19%	1125
1.12	650	0%	591

Tabla 2: valores obtenidos en simulaciones ante una entrada escalón de -10 con compensación a la saturación

Experimento

Una vez ya obtenidos los parámetros necesarios para la configuración de nuestro sistema, procedimos, inicialmente, a ver cómo funcionaba el módulo provisto en la cátedra para control PID. A partir de esto, visualizamos que existía un código de electroventilador utilizado anteriormente en la materia y procedimos a readaptar dicho código con el fin de adecuarlo a la heladera.

Para esto, hubo que modificar la función de visualización de datos y algunos parámetros como, por ejemplo, salida máxima y mínima; y topes de integración.

Asimismo, planteamos la posibilidad de tener un segundo sensor de temperatura para contrastar en tiempo real contra la temperatura ambiente, ya que nuestro sistema depende mucho de dicha temperatura como fue mencionado en el primer trabajo de laboratorio.

Para esto, conectamos un segundo sensor en paralelo al primero y ejecutamos un código en el Arduino para poder iterar sobre ellos y obtener la dirección de memoria, con el fin de poder leer cada sensor de temperatura sabiendo su posición.

Hecho esto, procedimos a iniciar la experiencia de laboratorio.

Primera Experiencia

Como se mencionó anteriormente, inicialmente conectamos el segundo sensor de temperatura para tener una referencia de la temperatura ambiente en tiempo real, para luego cargar el código con el módulo provisto por la cátedra y tomando como referencia el código del electroventilador que se utilizó en otras oportunidades.

Una vez conectado esto y cargado el código, procedimos a efectuar las primeras mediciones preliminares. En este punto, nos surgieron dos grandes dificultades:

- A medida que la temperatura se iba acercando a nuestra temperatura objetivo, la corriente eléctrica disminuía drásticamente y la tensión de alimentación aumentaba de forma excesiva, llegando incluso a ser más del doble.
- A su vez, también a medida que la temperatura se acercaba al objetivo, el PWM suministrado al sistema, hacía que la tensión de salida no sea la suficiente para alimentar el cooler, por lo que el mismo se apagaba y quedaba con aproximadamente 1 °C de diferencia contra la temperatura deseada. Por ejemplo, si queríamos tener 15 °C como objetivo, solamente se acercaba hasta los 16 °C.

Segunda Experiencia

De las dificultades presentadas en la primera experiencias, pudimos plantear las siguientes tareas a realizar:

- Separar la conexión entre el ventilador y la celda de peltier, para que no se apague cuando el voltaje entregado sea menor, de lo que necesita el ventilador para prender.
- Asegurar que el voltaje no varía en exceso.

Replanteo de las conexiones del circuito:

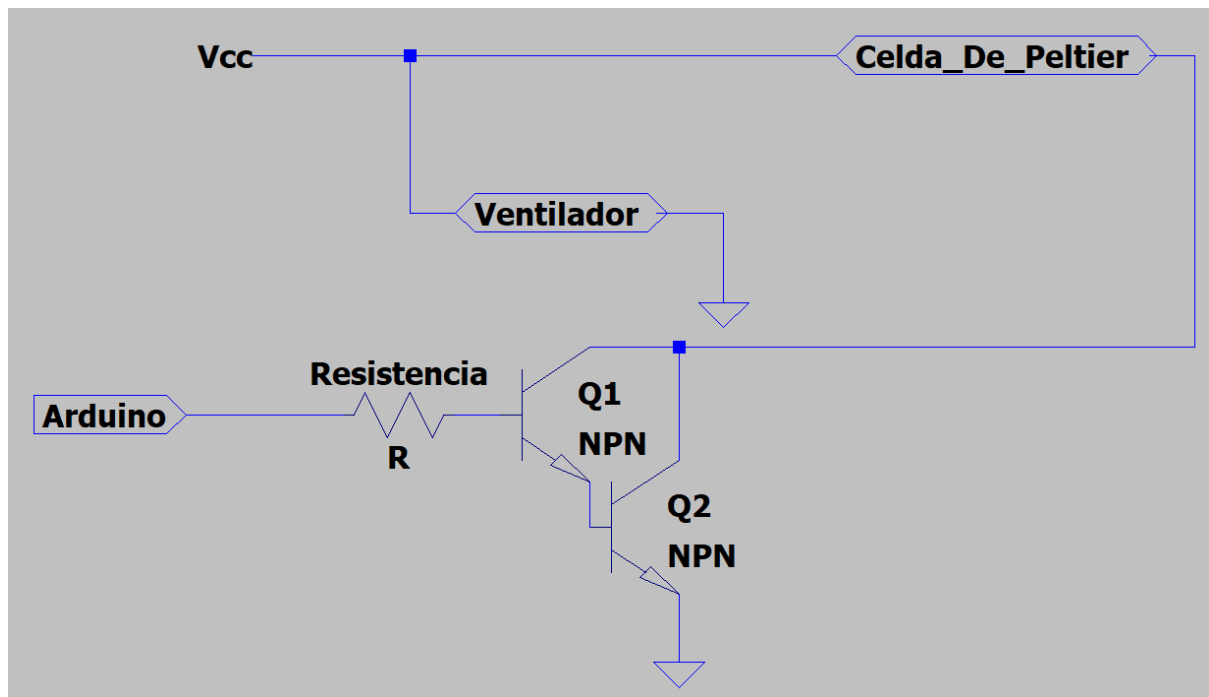


Fig 8. Elementos utilizados en nuestro circuito

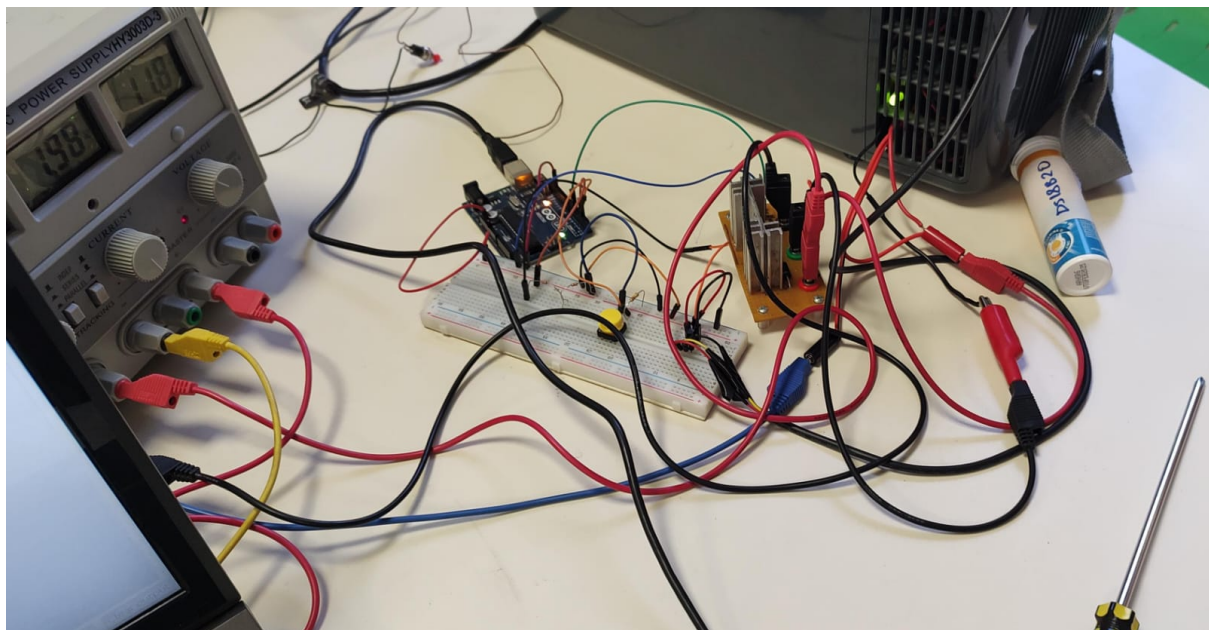


Fig 9. Circuito armado en la realidad.

Separamos las conexiones entre la celda de Peltier y el cooler que se encarga de quitar el calor del disipador, conectando la celda al transistor en configuración Darlington como se encontraba anteriormente y el cooler en paralelo al circuito conectado directamente a la fuente, ya que, cuando la tensión que administrábamos al circuito no era suficiente, se apagaba el cooler.

Luego conectamos un multímetro a nuestra fuente para tener una medición de confianza en cuanto a la cantidad de voltaje que está entregando.

Durante estos cambios, realizamos activaciones del sistema, por lo que el valor de las mediciones iniciales no concordaba con la temperatura ambiente. Para esto, decidimos calentar con la mano la zona del sensor, para que tenga una temperatura inicial cercana a la del ambiente.

Pero esto generó unas perturbaciones que no tiene sentido tomar en cuenta, por eso se ignoran los primeros 25 segundos, aunque los mismos fueron graficados para verificar el comportamiento.

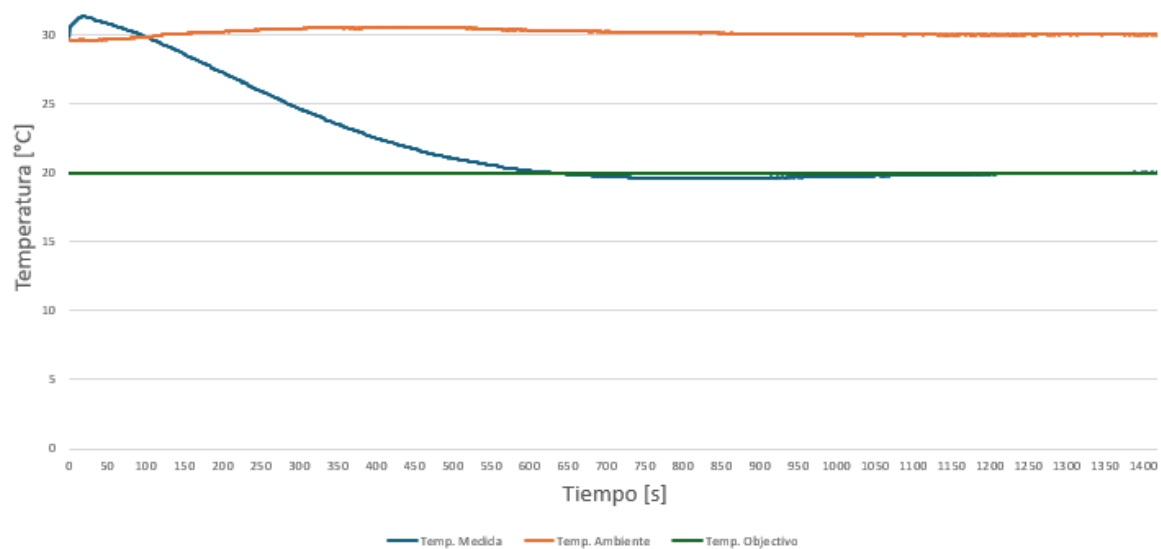


Fig 10. Gráfico de medición del sistema

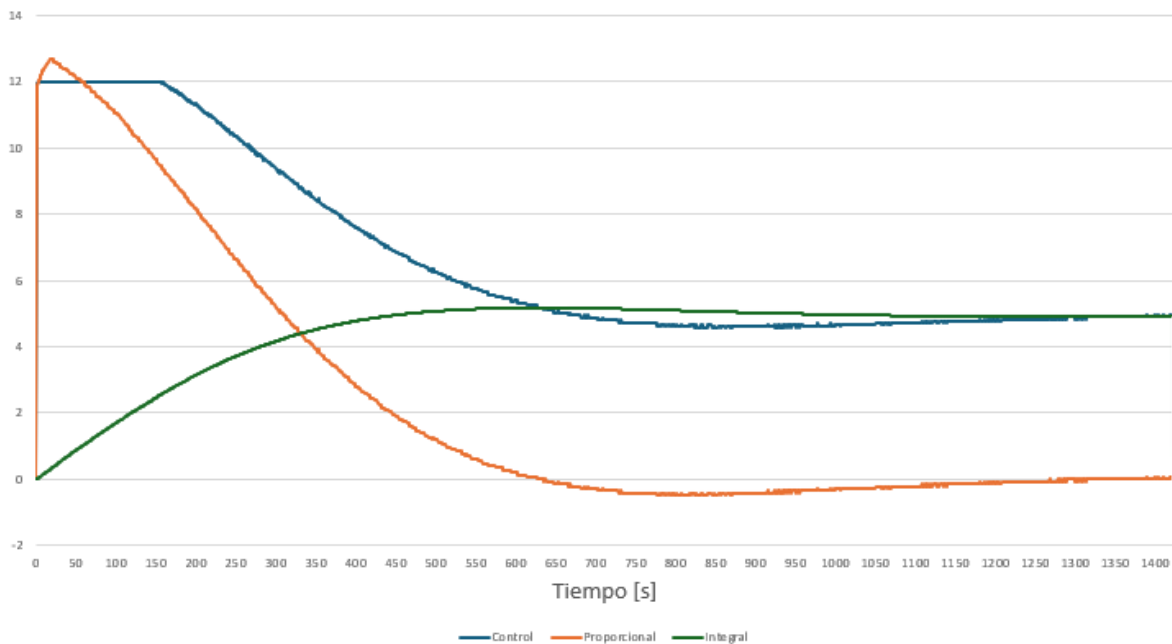


Fig 11. Gráfico de las acciones de control.

Según las mediciones tomadas, se alcanzó los 20°C a los 627 segundos, teniendo un sobrevalor de enfriamiento, ya que llegó a los 19,62°C dando un valor de sobrevalor de 3,45%

$T_i \rightarrow 31^{\circ}\text{C} \rightarrow$ temperatura inicial

$T_f \rightarrow 19.62^{\circ}\text{C} \rightarrow$ temperatura de enfriamiento máxima

$T_o \rightarrow 20^{\circ}\text{C} \rightarrow$ temperatura objetivo

$$(T_i - T_f) * \frac{100 \%}{T_i - T_o} = (31^{\circ}\text{C} - 19.62^{\circ}\text{C}) * \frac{100 \%}{31^{\circ}\text{C} - 20^{\circ}\text{C}} = 3,45\%$$

Luego de esto, decidimos hacer la prueba con una botella para ver cómo variaba el sistema y la respuesta del mismo ante una situación real, ya que hasta el momento, hicimos la prueba en vacío, por lo cual, obtuvimos la siguiente medición:

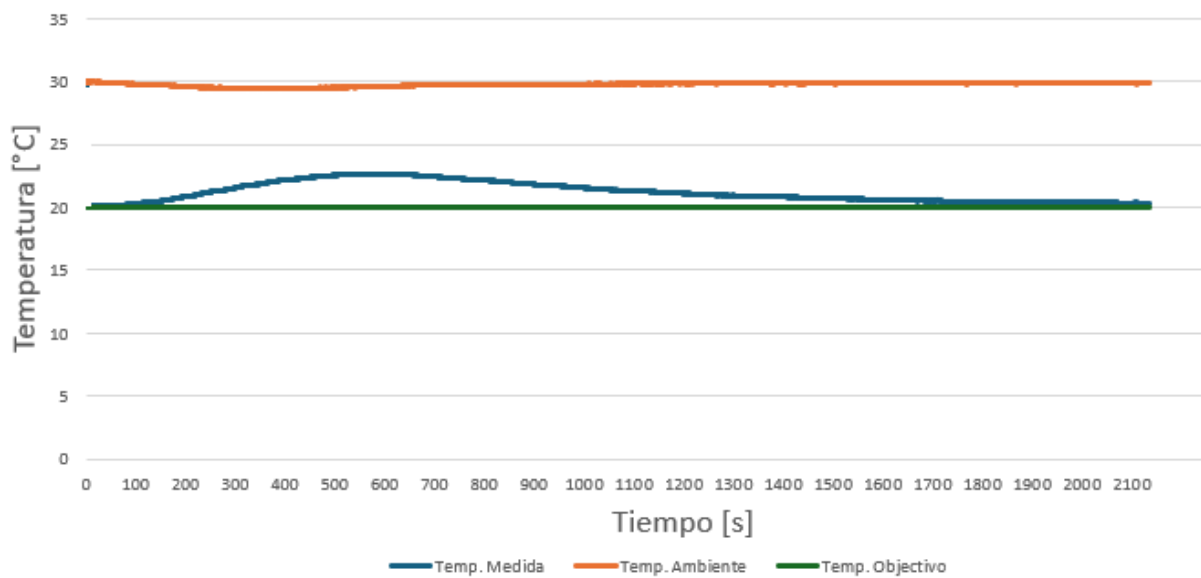


Fig 12. Gráfico de medición del sistema con una botella de agua.

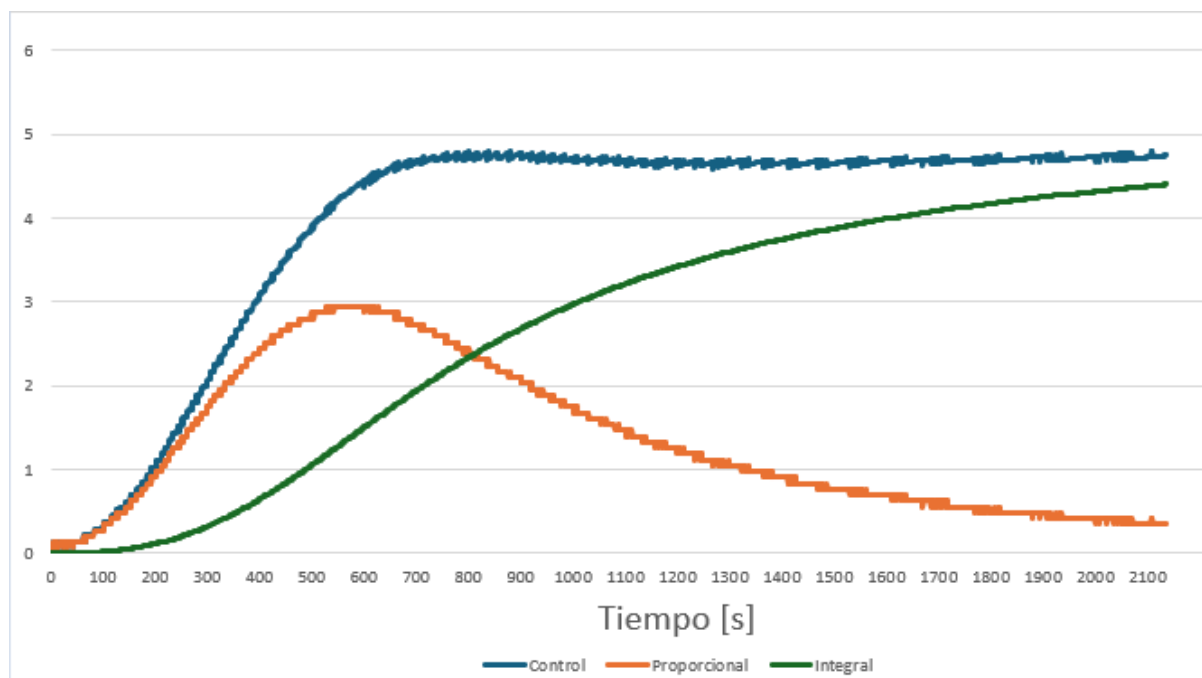


Fig 13. Gráfico de las acciones de control del sistema con una botella de agua.

Aquí podemos observar que, al momento de poner la botella, el interior de la heladera comienza a elevar su temperatura hasta encontrar una estabilidad térmica con la temperatura de la botella con el agua. A partir de ahí, una vez que la temperatura llega a su máximo, comienza nuevamente la disminución de esta, para lo cual, observamos que el tiempo aumenta drásticamente, lo cual tiene relación con la capacidad calórica del objeto que metimos dentro.

	Simulación	Realidad 
Sobrevalor	0%	3,45%
Tiempo de asentamiento [seg]	591	498

Tabla 3 -

Conclusiones

En conclusión, el experimento llevado a cabo para implementar un control proporcional-integral en el enfriamiento de una heladera mediante una Celda de Peltier ha arrojado resultados altamente positivos. Comparado con el control on-off, el enfoque proporcional-integral demostró ser más efectivo, proporcionando resultados notoriamente más precisos.

Las mediciones y la experiencia práctica coincidieron en gran medida con las simulaciones realizadas en SciLab, aunque se observaron pequeñas diferencias. Específicamente, se identificó un ligero sobrevalor en la realidad, pero, de manera alentadora, el tiempo de asentamiento se redujo. Estas discrepancias resaltan la importancia de la experimentación práctica y la validación empírica de los modelos teóricos.

A pesar de haber modelado el sistema como uno de primer orden, los resultados obtenidos fueron altamente satisfactorios. Este éxito respalda la eficacia del enfoque de control proporcional-integral en el contexto del enfriamiento de la heladera, subrayando la capacidad del sistema para responder de manera precisa a las variaciones de temperatura. En resumen, la combinación de la teoría, la simulación y la práctica experimental ha permitido avanzar significativamente en la comprensión y mejora del control automático en este contexto específico.

Bibliografía

Caporaletti, G., & Castellano, J. M. (2019). *MODELIZACIÓN DE SISTEMAS FÍSICOS EN 1º ORDEN*. Buenos Aires: UNDAV.

Caporaletti, G., & Castellano, J. M. (2021). *Introducción a sistemas de control automático*. Buenos Aires: UNDAV.

Caporaletti, G., & Castellano, J. M. Aproximación de MacLaurin. Buenos Aires: UNDAV.

Freire, G. A., D'Angiolo, F., & Kwist, I. (2022). *Métodos Matemáticos de la Ingeniería*. Buenos Aires: UNDAV.

Kuo, B. C. (1996). *Sistemas de Control Automático*. PRENTICE-HALL HISPANOAMERICANA, S.A.

Maxim Integrated Products, Inc. (July de 2019). *Data Sheet - DS18B20*. Obtenido de <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>

Anexo

Modelo de retardo de Maclaurin

Si consideramos la salida del sistema sin retardo como $y(t)$, entonces la salida $b(t)$ con retardo va a estar desplazada en un tiempo T_D (tiempo de retardo) con respecto a $y(t)$. Esto se puede representar de la siguiente forma:

$$b(t) = y(t - T_D)$$

Si a esta expresión le aplicamos Laplace con la propiedad de desplazamiento en el tiempo obteniendo lo siguiente:

$$\mathcal{L}\{f(t - T) \cdot u(t - T)\} = e^{-sT} \cdot F(s)$$
$$B(s) = e^{-T_D \cdot s} \cdot Y(s)$$

Y si lo expresamos como una función de transferencia obtendremos la expresión del retardo:

$$\frac{B(s)}{Y(s)} = e^{-T_D \cdot s}$$

En donde la entrada $Y(s)$ es igual a la salida del sistema sin retardo y $B(s)$ claramente será la nueva salida del sistema ya con retardo.

Muchas herramientas analíticas requieren funciones de transferencia racionales, ya que estas permiten estudiar la dinámica de la función de transferencia con sólo analizar los polos.

Debido a este motivo lo que se hace es aproximar el retardo $e^{-T_D \cdot s}$ mediante una expresión racional. Una de las formas de hacerlo es a través de la serie de Maclaurin.

SERIE DE MACLAURIN:

$$F(s) = \frac{F(0)}{0!} \cdot s^0 + \frac{F'(0)}{1!} \cdot s^1 + \frac{F''(0)}{2!} \cdot s^2 + \dots + \frac{F^{(n)}(0)}{n!} \cdot s^n = \sum_{n=0}^{\infty} \frac{F^{(n)}(0)}{n!} \cdot s^n$$

Aplicada al retardo y utilizando solo 3 términos:

$$e^{-T_D \cdot s} = \frac{1}{1 + T_D \cdot s + \frac{T_D^2 \cdot s^2}{2}}$$

Esta aproximación no es válida si el tiempo de retardo T_D es muy grande.

Para utilizar este retardo en nuestro modelo hay que recordar que la entrada al retardo tiene

el mismo valor que la salida el mismo, pero sin el desplazamiento en el tiempo

$$\frac{B(s)}{Y(s)} = \frac{1}{1 + T_D * s + \frac{T_D^2 * s^2}{2}}$$

Por lo tanto, la entrada Y(s) del retardo será la salida de la planta de nuestro sistema que teníamos implementado sin retardo.



Código Arduino del sensor de temperatura

Utilizamos el siguiente código para hacer las mediciones:

```
#include <OneWire.h>
#include <DallasTemperature.h>

OneWire ourWire(2);           //Se establece el pin 2 como bus
OneWire

DallasTemperature sensors(&ourWire); //Se declara una variable u objeto
para nuestro sensor

void setup() {
  Serial.begin(9600);
  sensors.begin(); //Se inicia el sensor
}

void loop() {
  sensors.requestTemperatures(); //Se envía el comando para leer la
temperatura
  float temperatura = sensors.getTempCByIndex(0); //Se obtiene la
temperatura en °C

  Serial.print("Temperatura= ");
  Serial.print(temperatura);
  Serial.println(" °C");
  delay(500);
}
```

Para poder obtener las direcciones de memoria, usamos el siguiente código:

```
#include <OneWire.h>

OneWire ourWire(2);           //Se establece el pin 2 como bus
OneWire
```

```

void setup(void) {
    Serial.begin(9600);
}

void loop(void) {
    byte addr[8];
    Serial.println("Obteniendo direcciones:");
    while (ourWire.search(addr)) {
        Serial.print("Address = ");
        for( int i = 0; i < 8; i++) {
            Serial.print(" 0x");
            Serial.print(addr[i], HEX);
        }
        Serial.println();
    }

    Serial.println();
    ourWire.reset_search();
    delay(2000);
}

```